

Lecture 4: Atomic Data Types

Laboratory: Coding - Accounting, Finance and Business Consulting

Python Statements

- A Python program (a.k.a. script when 'short') is a textual file
 - Made of **statements** evaluated by the Python interpreter from top to bottom
- The notebooks we use in this course are more complex documents
 - Python statements are written in code cells that you can execute in the order you prefer
- **Python statements**
 - An instruction that the Python interpreter can execute
 - Made of **Variables**, **Literals** and **Expressions**.

Variables

A variable is a container (storage area) to hold data (Tom's boxes)

To create a variable, you just name it and **assign** it a value with `=`

```
var_name = value
```

- Once a variable is assigned a value
 - It refers to the assigned value
 - It can be manipulated and acted upon (e.g., in `print(variable)`)

Variable Names

Variable names have their own rules:

- Variable names must start with a letter or underscore `_`
 - Can not start with a number
- Variable names must be composed of letters, numbers, or underscores
 - Can not contain special characters such as `!`, `@`, `###`, `$`, `%`, etc.
- Variable names are case sensitive

Variable Names

You cannot use reserved words as variable names / identifiers, e.g.,

```
import, True, False, or, and, not, else, if, elif, for, while,
continue, pass, break, is, in, class, return, finally, None,
lambda, def, from, nonlocal, del, global, with, as, try, yield,
assert, except, raise
```

```
In [1]: import = 0
```

```
Cell In[1], line 1
    import = 0
      ^
SyntaxError: invalid syntax
```

Variable Names

Examples of admissible variable names: `classroom_data_2`
`parliamentSpeeches14` `_growthRates`

- It is very important to choose informative names for your variables
 - It will help you and others understand your code when you read it again
 - Also prevents namespace hiccups
- Two most common naming conventions:
 - `classroom_data_2` uses the **snake case** convention
 - `parliamentSpeeches14` uses the **camel case** convention
 - Can also be combined to be more informative
 - Example `popIncome02Mar23_notClean`, `IDfirmsTextile_emilia_22`

Literals

- Literals are representations of fixed values
 - Can be of different type: Numbers, strings, booleans etc.
 - 'Hello, World!', 12, 23.0, 'C', etc.
- Literals cannot be assigned any value
 - When one is placed to the left of `=`, the interpreter raises that syntax error.

```
In [2]: 2 = "Ciao"
```

```
Cell In[2], line 1
    2 = "Ciao"
      ^
SyntaxError: cannot assign to literal
```

Expressions

- An expression can be composed by a sequence of:
 - Literals

- Variables
- Functions/operators such as `print()` or `+`
- This allows to store and manipulate data

Expressions / Evaluation and Data Types

- Expressions are evaluated differently depending on the data type of its arguments
 - The functions/operators behave differently depending on the type of the arguments
 - `3 + 4` is asking the computer to do a very different operation than `'Data' + 'Analysis'`
- Now that we know the basic components of an expression, we can start introducing the most common distinctions among data types

Data Types Characteristics

- **Scalar vs Non-scalar**
 - **Scalar:** Represents a single value
 - Simple so faster to process and manipulate
 - **Non-scalar:** Represents a collection or composite of multiple values
 - Can represent more complex data structures

Data Types Characteristics

- **Mutable vs Immutable**
 - **Mutable:** The value can be changed after creation
 - Provides flexibility in data manipulation, allowing for modifications, additions, and deletions
 - **Immutable:** Once created, the value cannot be changed
 - Ensures data integrity, especially when passed between functions or threads, preventing unintentional modifications

Data Types Characteristics

- **Ordered vs Unordered**
 - **Ordered:** Maintains a specific sequence or arrangement of elements
 - Predictable element arrangement allows for indexed access and sequence-based operations
 - **Unordered:** Does not maintain any specific sequence of elements
 - Typically faster for certain operations like membership tests and insertions since it doesn't need to maintain order

Built-in Data Types in Python

Type	Description	Purpose	Scalar	Mutability	Order
<code>int</code>	Integer number	Representing whole numbers	Scalar	Immutable	-
<code>float</code>	Floating-point number	Representing real numbers	Scalar	Immutable	-
<code>bool</code>	Boolean value	Representing True or False	Scalar	Immutable	-
<code>None</code>	Represents absence of value	Indicating a lack of value	Scalar	Immutable	-
<code>str</code>	Sequence of characters	Representing text	Non-scalar	Immutable	Ordered (sequence)
<code>tuple</code>	Immutable ordered collection of items	Grouping related data together	Non-scalar	Immutable	Ordered (sequence)
<code>list</code>	Mutable ordered collection of items	Storing items in a changeable sequence	Non-scalar	Mutable	Ordered (sequence)
<code>set</code>	Unordered collection of unique items	Removing duplicates and membership tests	Non-scalar	Mutable	Unordered
<code>dict</code>	Unordered collection of key-value pairs	Associative mapping from unique keys to values	Non-scalar	Mutable	Unordered

Types in this Lecture - Scalars and Strings

We will have a look at the following scalar types:

Type	Description	Purpose	Scalar	Mutability	Order
<code>int</code>	Integer number	Representing whole numbers	Scalar	Immutable	-
<code>float</code>	Floating-point number	Representing real numbers	Scalar	Immutable	-
<code>str</code>	Sequence of characters	Representing text	Non-scalar	Immutable	Ordered (sequence)
<code>bool</code>	Boolean value	Representing True or False	Scalar	Immutable	-
<code>None</code>	Represents absence of value	Indicating a lack of value	Scalar	Immutable	-

- Date and time handling managed by `datetime` module (not built-in)

Numeric Types in Python

- Primary types for numbers:
 - `int` integers
 - sequences of digits
 - `float` floating-point numbers
 - sequences of digits with a single `.` in between

`int`

- `int` can store arbitrarily large numbers.

```
In [3]: big_int = 26254519291092456596965462913230729701102721
print(big_int)
```

26254519291092456596965462913230729701102721

`float`

- `float` is the double-precision type in Python (64 bits)
 - Values are stored in groups of 64 bits, each of which is a series of 64 zeros and ones
 - Can represent numbers in the range 10^{-308} to 10^{308}
 - The precision is about 16 digits
 - Can also be written in scientific notation, e.g., `1.0e-10` or `1.0e10`

```
In [4]: val_f = 7.243
val2_f = 6.78e-5
print(f"val_f is: {val_f}, val2_f is: {val2_f}")
```

val_f is: 7.243, val2_f is: 6.78e-05

Arithmetic Operations

- Python supports the usual arithmetic operations for numeric types
 - Done using operators
 - Operators are nothing special, just function with a special syntax for:
 - Recurring operations
 - Operations with a heritage
 - The arithmetic ones take the form that is common in math

Arithmetic Operations

The `+` operator between two numeric types computes their addition.

The `-` operator between two numeric types computes their subtraction.

The `*` operator between two numeric types computes their multiplication.

The `/` operator between two numeric types computes their division, returning their quotient as float number.

The `//` operator between two numeric types computes their integer division, returning their quotient as integer number.

The `%` operator between two numeric types computes their module, returning their remainder as integer number.

The `**` operator between two numeric types computes the exponentiation of the left-side by the power of the right-side number.

```
In [5]: a = 3
        b = 2

        # Display 'a' and 'b'
        print(f"The number a is {a}, the number b is {b}")

        # Display the sum of 'a' and 'b'
        print(f"a + b = {a + b}")

        # Display the difference between 'a' and 'b'
        print(f"a - b = {a - b}")

        # Display the product of 'a' and 'b'
        print(f"a * b = {a * b}")

        # Display the division of 'a' by 'b'
        print(f"a / b = {a / b}")
```

The number a is 3, the number b is 2

a + b = 5

a - b = 1

a * b = 6

a / b = 1.5

Integer division yields a float if not a whole number

```
In [6]: a / b ### 1.5
```

```
Out[6]: 1.5
```

The Floor division , module and exponentiation operator

The floor division operator" `//` only returns the integer part of the quotient.

```
In [7]: a // b
```

```
Out[7]: 1
```

The `%` operator between two numeric types computes their module, returning their remainder as an integer

The `**` operator between two numeric types computes the exponentiation of the left-side (base) by right-side (exponent) number

```
In [8]: a = 3
c = 4
d = 5
print(c % a)
print(d % a)
print(a ** c)
```

```
1
2
81
```

Augmented Assignment Operators

Some operators are combined with the assignment operator `=` to form a new operator

Python offers several augmented assignment operators:

Operator	Type	Example	Equivalent
<code>+=</code>	Addition	<code>a += 2</code>	<code>a = a + 2</code>
<code>-=</code>	Subtraction	<code>a -= 2</code>	<code>a = a - 2</code>
<code>*=</code>	Multiplication	<code>a *= 2</code>	<code>a = a * 2</code>
<code>/=</code>	Division	<code>a /= 2</code>	<code>a = a / 2</code>
<code>**=</code>	Power	<code>a **= 2</code>	<code>a = a ** 2</code>

Numerical Example with `+=`

```
In [9]: a = 10
a += 7.5
print(a)
```

```
17.5
```

This is equivalent to `a = a + 7.5`, but more "Pythonic".

Floating-Point Precision Issues

- Integers represent exactly whole numbers (much less demanding for the computer)

- Floating point numbers that are (very precise) **approximations** of a number
 - Representing number using powers of 2
 - Since doubles are approximations, comparisons of equality sometimes return unexpected results

```
In [10]: x_precision = 0.1
x_precision += 0.05
print(x_precision)
print(x_precision == 0.15)
```

```
0.15000000000000002
False
```

The number `0.1` and `0.05` are stored with approximations, and their sum is not exactly `0.15`.

Check True Value

- Already from printing `x_precision` you could see that something was off
 - To further inspect the true value, you can use the `format` function (more on that later)

```
In [11]: print(format(x_precision, ".50f"))
print(format(.15, ".50f"))
print(format(.1+.05, ".50f"))
```

```
0.15000000000000002220446049250313080847263336181641
0.14999999999999999444888487687421729788184165954590
0.15000000000000002220446049250313080847263336181641
```

This is why `==` returns `False`

How to compare floats then?

Compare Differences Using Tolerance

You could write this check yourself using a tolerance

- Two numbers equal if the difference between the two is small enough

```
In [12]: tol = 1e-9
(x_precision-0.15)<tol
```

```
Out[12]: True
```

Compare Differences using `isclose`

the `math` package provides an `-equality check with tolerance-` function already implemented

```
In [13]: import math
         math.isclose(x_precision, 0.15)
```

Out[13]: True

```
In [14]: math.isclose?
```

The function `isclose()` takes three arguments: - The first two are the values to compare - The third is the tolerance, i.e., the maximum difference allowed between the two values for them to be considered equal

- If you do not specify the tolerance, it will be set to a very small number (1e-9)

Integers are Exact

With integers, there are **no precision issues**

The `==` operator always works as expected

```
In [15]: int1 = 5
         int2 = 3+2
         print(int1 == int2) # True
```

True

Strings

- Strings are not atomic types
 - They are ordered sequences of characters
 - We are introducing them here because they are one of the fundamental types in Python
 - Allows also to introduce some concepts/functions that will be also useful later
 - We are going to see more about compound data types in the next lecture
- Python offers a rich set of operations and functions for working with strings
- The built-in string type is `str` (Unicode encoding)
- Strings can be defined using either single `'` or double `"` quotes.

```
In [16]: print('a string is a string no matter how many empty spaces are in it')
         type('a string is a string no matter how many empty spaces are in it')
```

a string is a string no matter how many empty spaces are in it

Out[16]: str

```
In [17]: print("if you have a single ' inside a string, encapsulate your string withi  
if you have a single ' inside a string, encapsulate your string within doubl  
e strings
```

Operations on Strings: Concatenation and repetition

- Strings can be concatenated with the `+` operator, and repeated with `*`
 - Clearly, the `+` and `*` operations behave differently on strings and on numbers

```
In [18]: print("5" + "3")  
print("Data " * 2)
```

```
53  
Data Data
```

String Example with `+=`

```
In [19]: forename = "John"  
surname = " Swanson"  
forename += surname  
print(forename)
```

```
John Swanson
```

Accessing Elements in Sequences (Characters in Strings)

- Sequences: Lists, tuples, and **strings** are ordered
 - Each element in these data types is indexed by an integer
 - You can access each element using the indexing operator `[]`

You can:

- **Index** `sequence[index]` to extract individual elements
- **Slice** `sequence[start:end]` to get sub-sequence starting from index `start` and ending at index `end-1`
- **Striding** `sequence[start:end:step]` to get sub-sequence starting from index `start`, in steps of `step`, ending at index `end-1`

Indexing

Indexing is performed via square brackets and an integer

- The index represents the position of the element to retrieve
- **Indexing starts at 0**
- Negative indices start from the end of the string

Non-negative indexes

- Non-negative indexes are 0 or positive integers
- 0 indicates the first element
- An increase in the index moves one position from left to right

Non-negative index:	0	1	2	3	4	5	6	7
Element:	H	e	r	m	i	o	n	e

```
In [20]: slides_string = "Hermione"
print(slides_string[0])
print(slides_string[1])
print(slides_string[5])
```

```
H
e
o
```

Length of a String - The number of characters

You can count the number of characters in a string using the function `len`

- Unlike indexing that starts at 0, counting starts at 1
 - If you want to retrieve the last character in a string you need to index at `len(str)-1`
 - If you try to retrieve at an index that is not there, an exception will be raised

```
In [21]: print(len(slides_string))
print(slides_string[(len(slides_string)-1)])
print(slides_string[len(slides_string)])
```

```
8
e
```

```
-----
IndexError                                Traceback (most recent call last)
Cell In[21], line 3
      1 print(len(slides_string))
      2 print(slides_string[(len(slides_string)-1)])
----> 3 print(slides_string[len(slides_string)])

IndexError: string index out of range
```

Negative indexes

- Negative indexes are negative integers
- -1 indicates the last element
- A decrease in the index moves one position from right to left

Element:	H	e	r	m	i	o	n	e
Negative index:	-8	-7	-6	-5	-4	-3	-2	-1

```
In [22]: slides_string = 'Hermione'
#print(slides_string[-1])
#print(slides_string[-2])
print(slides_string[-8])
print(slides_string[-len(slides_string)])
```

H
H

Slicing

- Slicing can be performed on a variety of objects (strings, lists, NumPy arrays)
 - It is a way to extract a subset of elements from an object
- Slice notation uses `start:end` format within square brackets `[]`
 - Start index is included, stop index is excluded.
 - Result length: `stop - start`.
- If you omit one of the parameters uses defaults for `start` and `stop`:
 - `start` defaults to `0`
 - `stop` defaults to `len(string)-1`

```
In [23]: print(slides_string[1:6])
print(slides_string[:6])
print(slides_string[6:])
```

ermio
Hermio
ne

Striding

Striding is performed via:

- Square brackets and double colons separating three fields
 - A field can be filled with either an integer or nothing
 - The left-most integer represents the start index (included)
 - the central integer represents the end index (excluded)
 - the right-most integer represents the stride
 - The number of chars to move forward at each step
 - A negative number move backwards, eg., -1 reverses the order of the elements

Element:	H	e	r	m	i	o	n	e
----------	---	---	---	---	---	---	---	---

Then `word[::-1]` is `word` read backwards:

Element:	e	n	o	i	m	r	e	H
----------	---	---	---	---	---	---	---	---

```
In [24]: print(slides_string[::2])
print(slides_string[2::2])
print(slides_string[::-1])
```

```
Hrin
rin
enoimreH
```

PRACTICE 1

Take the string 'What a lovely day of October!' and extract and print the following characters:

1. First and last
2. Second and second to last
3. Third and third to last
4. Second to second to last
5. First to last, only odd indexes
6. Second to second to last, only indexes multiple of 3
7. Second to last to second, only even indexes

```
In [ ]:
```

Assigning to Slices

In general you can assign values using slice notation

- But this cannot be done on immutable objects like strings, tuples, and NumPy arrays (discussed later)
- Python strings are immutable so can't be modified directly
- If we try to assign to an indexed position, we get an error

```
In [25]: slides_string[3:5] = [6, 3]
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[25], line 1
----> 1 slides_string[3:5] = [6, 3]

TypeError: 'str' object does not support item assignment
```

but this works on lists (we'll see them in a short while)

```
In [26]: n_list = [1, 0, 6, 3, 6, 3, 2, 7]
n_list[3:5] = [4, 4]
n_list
```

```
Out[26]: [1, 0, 6, 4, 4, 3, 2, 7]
```

Immutability of Python Strings

- If you need to modify components of a string you can't do it "in place" (in the same variable)
 - You need to modify the original object and then re-assign it (possibly to a variable with the same name, overwriting the original)
 - To use the original string as a starting point, you can use the `replace` method
 - Remember that a method is a function that is attached to an object (usually done for operations that are common for that object type)
 - In this case we want to apply the `replace` method to a string so the syntax will be `stringname.replace()`

```
In [27]: ###just modify the string, without changing the original string
a = "this is a string"
print(a.replace("string", "longer string"))
print(a) ###Original string remains unchanged after modification.
###modify the original string and save it to a new variable
b = a.replace("string", "longer string")
print(b)
###modify the original string and re-assign it to the same variable (overwriting)
a = a.replace("string", "longer string")
print(a)
```

```
this is a longer string
this is a string
this is a longer string
this is a longer string
```

Multi-Line Strings

- Multiline strings can be created using triple quotes `'''` or `"""`
 - Include line breaks `\n` as part of the string

```
In [28]: c = """This is a longer string that
spans multiple lines"""
print(c)
c.count('\n')
```

```
This is a longer string that
spans multiple lines
```

```
Out[28]: 1
```

Given that line breaks are just `\n`, we can get to a single line by replacing them with a space or nothing (but be aware of the spaces left at beginning/end)

```
In [29]: d = c.replace('\n', ' ')
d
```

```
Out[29]: 'This is a longer string that spans multiple lines'
```

String Formatting and Concatenation - f-Strings

- Python allows you to combine strings and format them in sophisticated ways
- Several ways to do this, but the most flexible is to use f-strings
 - introduced in Python 3.6
 - f-strings are prefixed with `f` or `F` and can contain expressions in curly braces `{}` that will be replaced with their values

```
In [30]: name = "John"
age = 31
greeting = f"Hello, my name is {name} and I am {age} years old."
greeting
```

```
Out[30]: 'Hello, my name is John and I am 31 years old.'
```

Inside the curly braces you can manipulate the variable, so to avoid storing results that are only needed for presentation

```
In [31]: greeting_young = f"Hello, my name is Alice and I am {round(age/2)} years old"
greeting_young
```

```
Out[31]: 'Hello, my name is Alice and I am 16 years old.'
```

Formatting Strings

When reporting results, it is often useful to control the appearance

- You can use the `format` method of strings to control how values are printed
- Also `fstrings` allow to perform variable formatting

Formatting Decimal Digits with f-Strings in Python

- When you print a float, Python displays a certain number of decimal digits by default
- Using f-strings, you can control the number of decimal places by appending a format specification.
- Syntax: `{expression:--num_of_digits--f}`
 - `expression` : The value you want to format.
 - `num_of_digits` : Number of decimal digits to display.

```
In [32]: pi = 3.14159265359
formatted_pi = f"Value of Pi rounded to 2 decimal places: {pi:.2f}"
formatted_pi_4_digits = f"Value of Pi rounded to 4 decimal places: {pi:.4f}"
print(formatted_pi)
print(formatted_pi_4_digits)
```

```
Value of Pi rounded to 2 decimal places: 3.14
```

```
Value of Pi rounded to 4 decimal places: 3.1416
```

Other Main Formatting Options in f-Strings

The default is String Formatting

- `{variable:s}` : Formats the variable as a string.

```
In [33]: name = "Alice"  
f"Name: {name:s}"
```

```
Out[33]: 'Name: Alice'
```

Scientific Notation

- `{variable:e}` : Formats the variable in scientific notation.

```
In [34]: number = 1000000  
f"Scientific notation: {number:e}"
```

```
Out[34]: 'Scientific notation: 1.000000e+06'
```

Percentage Formatting

- `{variable:%}` : Formats the variable as a percentage.

```
In [35]: percent = 0.25  
f"Percentage: {percent:%}"
```

```
Out[35]: 'Percentage: 25.000000%'
```

Combining Format Codes

- You can combine multiple format codes for a single variable.

```
In [36]: number = 12.3456  
f"Combined: {number:.2%}"
```

```
Out[36]: 'Combined: 1234.56%'
```

- Clearly you can use a different formatting for different variables

```
In [37]: large_number = 123456789  
completion_rate = 0.987  
f"Large Number in Scientific Notation: {large_number:e}, Completion Rate: {c
```

```
Out[37]: 'Large Number in Scientific Notation: 1.234568e+08, Completion Rate: 98.7  
0%'
```


PRACTICE 3

1. Take the string "I am a string" and print it in uppercase, lowercase, and title case
2. Using fstrings print the following string: "My name is {name} and I am {age} years old." where `name` and `age` are variables that you define
3. Using fstrings print the following string: "The market share of {product} is {share}" where `product` and `share` are variables that you define. `product` should be capitalized and `share` should be formatted as a percentage with 2 decimal digits

Find

The find function allows to search for a substring inside a string

- Returns the int index where the given substring appears or `-1` if not found
- Searches the string left to right
- Is case sensitive
- Only reports the first occurrence

```
In [38]: string_postTab = "Padua,Italy,35100"
print(string_postTab.find('|'))
print(string_postTab.find(','))
```

```
-1
5
```

Split

Split divides a string into a list of string elements based on a "separator" parameter

- The default split parameter is whitespace (space, tab, newline)
- Returns the empty string if we have two separators next to each other (but whitespaces are grouped)
- Split is often used to parse information from strings

```
In [39]: print("Maria Rossi".split())
print("Maria Rossi".split())
```

```
['Maria', 'Rossi']
['Maria', 'Rossi']
```

```
In [40]: string_postTab.split(",")
```

```
Out[40]: ['Padua', 'Italy', '35100']
```

PRACTICE 2

Take the Comma Separated Values string "1, 2, 3, 4, 5, 6, 7, 8, 9, 10" and split it into its single values. Comma separated values are a common way to store data in a text file, where each line is a row and column values are separated by commas.

Lower/Upper

The lower, upper and capitalize functions do the expected.

```
In [41]: s = "Hello, Hermione. Bye."

print(s.lower())

print(s.upper())

print(s.capitalize())
```

```
hello, hermione. bye.
HELLO, HERMIONE. BYE.
Hello, hermione. bye.
```

Lower/Upper

Lower is often used when doing comparisons between multiple strings to avoid unwanted case-sensitive mismatch

```
In [42]: print("Marco" == "marco")
"Marco".lower() == "marco".lower()
```

```
False
```

```
Out[42]: True
```

Help for methods

To find the help for a method the syntax is `type.methodName?`

```
In [43]: print(type("Marco"))
str.lower?
```

```
<class 'str'>
```

Encoding - Unicode and ASCII

- As we already mentioned, text is represented on a computer using encodings
 - ASCII is the most basic (and old) encoding, using 7 bits to represent 128 characters
 - This implies that different languages assign different characters to the same bit patterns
 - Need to use the right encoding to display text correctly (italians use latin1, greeks use greek, etc.)

- Unicode is a superset of ASCII and supports over a million characters
 - Standard for representing characters in Python (str)
 - Represents text not in terms of bites but with code points (integers)
 - No longer need to worry about the underlying encoding
- Sometimes it happens that when you read a file, the encoding is not correct
 - You will see strange characters instead of the expected ones
 - You can specify the encoding when opening the file (e.g., `open('file.txt', encoding='utf-8')`)

Working with Booleans in Python

- The `bool` type is used to represent Boolean values
 - Can be either `True` or `False`
 - Used to store information on a value that can be in any of two states
 - True or False, but also Yes or No, On or Off, etc.
 - Behave like integers 1 and 0

The many ways of creating a Boolean - Directly

1. Directly stating `True` , `False`

```
In [44]: print(True)
         print(False)

         print(True*5)
         print(False*5)
```

```
True
False
5
0
```

The Many Ways of Creating a Boolean - Logical Operators

Booleans can arise from the combination of other booleans using the logical operators

- `not` Negation
- `and` Conjunction
- `or` Disjunction

There are many other logical operators but these are crucial to this course

To understand how Boolean expressions relate to logical operations, here are reported their "Truth Tables"

- `a` and `b` are two booleans

Truth Table of Negation:

a	not a
True	False
False	True

```
In [45]: #First Table
print(f"not True -> {not True}")
print(f"not False -> {not False}")
```

not True -> False

not False -> True

Truth Table of Conjunction:

a	b	a and b
True	True	True
True	False	False
False	True	False
False	False	False

```
In [46]: #Second Table
print(f"True and True -> {True and True}")
print(f"True and False -> {True and False}")
print(f"False and True -> {False and True}")
print(f"False and False -> {False and False}")
```

True and True -> True

True and False -> False

False and True -> False

False and False -> False

For Disjunction:

a	b	a or b
True	True	True
True	False	True
False	True	True
False	False	False

PRACTICE 4

Write down the Disjunction truth table using code

The "bit-wise" and "element-wise" Logical Operators

Similar **but different** logical operators are:

- `~`, `&`, and `|` that correspond to `not`, `and`, `or` but operate on each element of an object
- If used on Python's numeric formats they act on each 0 and 1 storing its content (bit-wise)
 - As we said already, all the data in a computer is stored as 0s and 1s on a computer
 - It is sometimes useful to consider each 0/1 as True/False and operate on each of them using logical operators
 - We won't use this functionality in this course (more useful for Computer Scientists)
- If used on Numpy's and Pandas data formats `~`, `&`, and `|` perform logical operations on each of the elements composing them (from bit-wise they become element-wise)
 - Very useful in data analysis
- Be mindful of choosing the right Logical Operator on the different data types
 - **It is easy to get them mixed up leading to errors**

The Many Ways of Creating a Boolean - Comparison Operators

Booleans can arise from the use of comparison operators

Let `@` be any of the below comparison operators. Then, if the comparison has a sense (there is an ordering) the expression `a @ b` is Boolean

- In other words, the result `a @ b` is either `True` or `False`
- Ordering is cardinality for numbers (which one is bigger), alphabetical for strings
 - For other types it might be not defined, or defined by the programmer arbitrarily

Operator	Example	Meaning	Result
<code>==</code>	<code>a == b</code>	Equal to	True if the value of <code>a</code> is equal to the value of <code>b</code> False otherwise
<code>!=</code>	<code>a != b</code>	Not equal to	True if <code>a</code> is not equal to <code>b</code> False otherwise
<code><</code>	<code>a < b</code>	Less than	True if <code>a</code> is less than <code>b</code> False otherwise
<code><=</code>	<code>a <= b</code>	Less than or equal to	True if <code>a</code> is less than or equal to <code>b</code> False otherwise
<code>></code>	<code>a > b</code>	Greater than	True if <code>a</code> is greater than <code>b</code> False otherwise

Operator	Example	Meaning	Result
>=	a >= b	Greater than or equal to	True if a is greater than or equal to b False otherwise

```
In [47]: print(f"5==3 -> {5==3}")
print(f"5!=3 -> {5!=3}")
print(f"7<7 -> {7<7}")
print(f"5>3 -> {5>3}")
print(f"5>=5 -> {5>=5}")
```

```
5==3 -> False
5!=3 -> True
7<7 -> False
5>3 -> True
5>=5 -> True
```

```
In [48]: string_1 = "Hello"
string_2 = "Hi"
#comparison in strings is defined lexicographically (i.e. by the alphabetical order)
print(f"{string_1} == {string_2} -> {string_1==string_2}")
print(f"{string_1} >= {string_2} -> {string_1>=string_2}")
print(f"{string_1} < {string_2} -> {string_1<string_2}")
```

```
Hello == Hi -> False
Hello >= Hi -> False
Hello < Hi -> True
```

The Many Ways of Creating a Boolean - `is` and `in`

- `is` checks if two variables refer to the same object
 - `is not` checks if two variables refer to different objects
- `in` checks if a value is present in a sequence
 - `not in` checks if a value is not present in a sequence

```
In [49]: print(f"e in {string_1} -> {'e' in string_1}")
print(f"e not in {string_2} -> {'e' not in string_2}")
```

```
e in Hello -> True
e not in Hi -> True
```

```
In [50]: p = [1, 2, 3]
q = p
print(f"p is q -> {p is q}")
```

```
p is q -> True
```

```
In [51]: r = [1, 2, 3]
print(f"p is not r -> {p is not r}")
```

```
p is not r -> True
```

The `None` type

- `None` is a versatile tool in Python for representing the absence of a value or a null state
- `is` and `is not` operators are commonly used to check for `None` .

```
In [52]: b = 5
b is not None
```

Out[52]: True

- `None` is also a common default value for function arguments.

```
def add_and_maybe_multiply(a, b, c=None):
    result = a + b
    if c is not None:
        result = result * c
    return result
```

Dates and Times using the `datetime` module

- Python has built-in `datetime` module for manipulating dates and times
- Provides `datetime`, `date`, and `time` types
- `datetime` type is the most commonly used

```
In [53]: from datetime import datetime, date, time
?datetime
```

```
In [54]: dt = datetime(2011, 10, 29, 20, 30, 21)
dt
```

Out[54]: `datetime.datetime(2011, 10, 29, 20, 30, 21)`

Extracting Date and Time Components

- Extract `date` and `time` objects from a `datetime` instance.
- Access individual components like `day`, `minute` .

```
In [55]: print(f"dt.date() -> {dt.date()}")
print(f"dt.time() -> {dt.time()}")
print(f"dt.day -> {dt.day}")
print(f"dt.minute -> {dt.minute}")
```

```
dt.date() -> 2011-10-29
dt.time() -> 20:30:21
dt.day -> 29
dt.minute -> 30
```

Formatting dates - Exporting dates to strings

- You can ask for the date-time information in the format you want
- The keys for the most common components are:
 - `%Y` for the year
 - `%m` for the month
 - `%d` for the day
 - `%H` for the hour
 - `%M` for the minute
 - `%S` for the second
- you can find all the components [here](#)

```
In [56]: print(f"dt.strftime('%Y-%m-%d %H:%M') -> {dt.strftime('%Y-%m-%d %H:%M')}")
print(f"dt.strftime('%Y/%m/%d %H.%M.%S') -> {dt.strftime('%Y/%m/%d %H.%M.%S')}")

dt.strftime('%Y-%m-%d %H:%M') -> 2011-10-29 20:30
dt.strftime('%Y/%m/%d %H.%M.%S') -> 2011/10/29 20.30.21
```

Parsing dates - Importing dates from strings

- Often dates are stored in string format
 - We need to parse them into `datetime` objects
 - We can use the function `strptime` to parse
 - Must provide the format of the date string
 - e.g., `%Y-%m-%d` for `2020-01-01` or `%d/%m/%Y` for `01/01/2020`

```
In [57]: date_strp_1 = datetime.strptime("20091031", "%Y%m%d")
date_strp_1.strftime("%Y-%m-%d %H:%M")
```

```
Out[57]: '2009-10-31 00:00'
```

```
In [58]: date_strp_2 = datetime.strptime("2009-11-30", "%Y-%m-%d")
date_strp_2.strftime("%Y-%m-%d %H:%M")
```

```
Out[58]: '2009-11-30 00:00'
```

```
In [59]: date_strp_3 = datetime.strptime("31122009", "%d%m%Y")
date_strp_3.strftime("%Y-%m-%d %H:%M")
```

```
Out[59]: '2009-12-31 00:00'
```

Manipulating `datetime` Objects

- You can edit each component of the `datetime` objects using the `replace`
 - Analogous to the `replace` method for strings

```
In [60]: print(f"dt.strftime('%Y-%m-%d %H:%M') -> {dt.strftime('%Y-%m-%d %H:%M')}")
dt_hour = dt.replace(minute=0, second=0)
print("dt_hour = dt.replace(minute=0, second=0)")
print(f"dt_hour.strftime('%Y-%m-%d %H:%M') -> {dt_hour.strftime('%Y-%m-%d %H:%M')}")
```



```
dt.strftime('%Y-%m-%d %H:%M') -> 2011-10-29 20:30
dt_hour = dt.replace(minute=0, second=0)
dt_hour.strftime('%Y-%m-%d %H:%M') -> 2011-10-29 20:00
```

- `datetime` objects are immutable; methods produce new objects.

Working with Time Deltas

- Subtracting two `datetime` objects gives a `datetime.timedelta`.

```
In [61]: delta = dt - dt_hour
delta
```

```
Out[61]: datetime.timedelta(seconds=1821)
```

- Adding a `timedelta` to a `datetime` gives a new shifted datetime.

```
In [62]: print(f"dt_hour.strftime('%Y-%m-%d %H:%M') -> {dt_hour.strftime('%Y-%m-%d %H:%M')}")
dt_original = dt_hour + delta
dt_2deltas = dt_hour + 2 * delta
print("dt_original = dt_hour + delta")
print("dt_2deltas = dt_hour + 2 * delta")
print(f"dt_original.strftime('%Y-%m-%d %H:%M') -> {dt_original.strftime('%Y-%m-%d %H:%M')}")
print(f"dt_2deltas.strftime('%Y-%m-%d %H:%M') -> {dt_2deltas.strftime('%Y-%m-%d %H:%M')}")
```

```
dt_hour.strftime('%Y-%m-%d %H:%M') -> 2011-10-29 20:00
dt_original = dt_hour + delta
dt_2deltas = dt_hour + 2 * delta
dt_original.strftime('%Y-%m-%d %H:%M') -> 2011-10-29 20:30
dt_2deltas.strftime('%Y-%m-%d %H:%M') -> 2011-10-29 21:00
```

Types and Casting in Python

- The type of an object can be checked with the `type()` function
- It is also possible to convert values from one type to another with the following functions

Function	Description
<code>bool()</code>	Converts an argument to a Boolean value
<code>float()</code>	Returns a floating-point object constructed from a number or string
<code>int()</code>	Returns an integer object constructed from a number or string
<code>str()</code>	Returns a string version of an object

- Not all the conversions are possible and some may lead to loss of information (**always check the output!**)
 - `int` of a `float` will truncate the decimal part

- `bool(0)` is False, but `bool` of any other number is True
- String conversion usually does what you expect, but not always (e.g., string of a list is not the list itself)
- `int` and `float` on a string
 - Will work if the content can be parsed as number
 - Otherwise they raise an error

```
In [63]: float_var = 5.8
char_var = "Ciao"
charInt_var = "5"
print(f"float_var = {float_var}")
print(f"type of float_var is {type(float_var)}")
print(f"int(float_var) -> {int(float_var)}")
print(f"bool(0) -> {bool(0)}")
print(f"bool(float_var) -> {bool(float_var)}")
print(f"int(charInt_var) -> {int(charInt_var)}")
print(f"int(char_var) -> {int(char_var)}")
```

```
float_var = 5.8
type of float_var is <class 'float'>
int(float_var) -> 5
bool(0) -> False
bool(float_var) -> True
int(charInt_var) -> 5
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[63], line 10
      8 print(f"bool(float_var) -> {bool(float_var)}")
      9 print(f"int(charInt_var) -> {int(charInt_var)}")
----> 10 print(f"int(char_var) -> {int(char_var)}")

ValueError: invalid literal for int() with base 10: 'Ciao'
```

Notes

Book references:

Ch.4 of [Python for Data Analysis](#)

Office Hours:

See on my [website](#)

Author: Duccio Gamannossi degl'Innocenti

Web: <https://www.dgdi.me/>

Mail: duccio.gamannossi@unipd.it

Homework

Numeric Types

Exercise: Exploring Numeric Types in Python

Objective: Understand and practice arithmetic operations, comparisons, and type conversions with Python's numeric data types (integers and floating point numbers).

Part 1: Basic Arithmetic Operations

1. Calculate the sum of `15` and `25`. Print the result.
2. Divide `20` by `3` and store the result as a floating point number. Print the result.
3. Compute the modulus (remainder) of `17` divided by `5`. Print the result.

Part 2: Numeric Comparisons

1. Check if `7` squared (7 raised to the power of 2) is greater than `50`. Print the result.
2. Compare if the square root of `81` is equal to `9`. (Hint: Use the `**` operator for square root). Print the result.

Part 3: Type Conversion and Rounding

1. Convert the floating point number `7.8` to an integer. Print the result.
2. Round the number `15.6` to the nearest whole number. Print the result.
3. Convert the integer `12` to a floating point number. Print the result.

Exercise: Understanding Floating Point Imprecision

Objective: Grasp the concept that floating point numbers in computers may not always hold exact values, leading to potential issues when comparing them directly for equality.

Task:

1. Calculate the expression `0.1 + 0.2`.
 2. Check if the result of the above calculation is equal to `0.3`. Print the result.
 3. Instead of direct comparison, check for equality within a small tolerance, say 10^{-9} , to handle the imprecision. Print this result.
-

Strings

Exercise: String Slicing, Indexing and Striding

Objective: Further your knowledge and practice with string slicing, indexing, and manipulation in Python.

Part 1: Indexing within a Phrase

1. Create a string `word` with the value "ELEPHANT".
2. Using indexing, display the third character of the string.
3. Using indexing, display the second-to-last character of the string.

Part 2: Slicing through Sentences

1. Create a string `sentence` with the value "Learning Python is fun!".
2. Using slicing, display the word "Learning".
3. Using slicing, display the word "fun" (without the exclamation mark).

Part 3: Manipulating Strings with Slicing

1. Create a string `alphabet` with the value "abcdefghijklmnopqrstuvwxyz".
 2. Using slicing, display the first half of the string.
 3. Using slicing, display every second letter from the string.
-

Exercise: Exploring F-String Literals and Formatting

Objective: Further understand the core features of f-string literals in Python.

Part 1: F-String Expressions

1. Create two variables, `x` with the value 7 and `y` with the value 8.
2. Using an f-string, display the result of `x` multiplied by `y` with the format: "7 times 8 is 56."
3. A possible other way to display the same result would be to compute the product of `x` and `y` and then use the `format` method on the string "7 times 8 is {}." Do you think this solution is better or worse than the one with f-strings? Why?

Part 2: F-String Precision

1. Create a variable `pi` with the value 3.141592653589793.
2. Using an f-string, format and display the value of `pi` rounded to 4 decimal places, like: "The value of pi is approximately 3.1416."
3. A possible other way to display only some digits would be to use the function `round` on the variable `pi`, so you think this solution is better or worse than the one with f-strings? Why?

Part 3: F-String with String Methods

1. Create a variable `greeting` with the value "hello".
2. Using an f-string, capitalize the greeting and display the message: "The greeting is: Hello."

Part 4: F-String with Type Conversion

1. Create a variable num with a string value "123".
 2. Using an f-string, convert the string to an integer and display twice its value, like:
"Twice the value is 246."
-

Booleans

Exercise: Exploration of Boolean Logic with Atomic Data Types

Objective: Continue to practice and reinforce understanding of Boolean operations using atomic data types in Python.

Part 1: Numeric Comparisons

1. Check if $9 - 4$ is equal to 5. Print the result.
2. Compare if 6×3 is greater than 20. Print the result.

Part 2: String Comparisons

1. Check if the first letter of the word "elephant" is "e". Print the result.
2. Check if the last letter of the word "tiger" is not "r". Print the result.

Part 3: Compound Boolean Evaluations

1. Evaluate if 3×3 is 9 and $4 + 4$ is 8. Print the result.
 2. Evaluate if $5 - 2$ is less than 2 or $6 \div 2$ is 3. Print the result.
-

Exercise: Advanced Boolean Logic with Atomic Data Types

Objective: Deepen your understanding of Boolean operations using only atomic data types in Python.

Part 1: Numeric Comparisons

1. Compare if the remainder of 15 divided by 4 is greater than 2. Print the result.
2. Check if 7.5 rounded to the nearest whole number is equal to 8. Print the result.

Part 2: String Operations with Boolean

1. Check if the length of the string "Python" is greater than the length of the string "Java". Print the result.
2. Check if the word "book" exists within the string "notebook". Print the result.

Part 3: Compound Boolean Logic

1. Evaluate if 5 is both greater than 3 and less than 8. Print the result.

2. Evaluate if `12` is either less than `10` or greater than `15` . Print the result.
 3. Using Boolean logic, check if `6 * 6` is `36` and if `7 - 2` is not `4` . Print the result.
-

Exercise: Exploring the `datetime` Module

Objective: Get familiar with basic operations and manipulations using Python's `datetime` module.

Part 1: Date and Time Retrieval

1. Import the `datetime` module.
2. Retrieve the current date and time. Print the result.
3. Retrieve just the current date. Print the result.

Part 2: Date Manipulation

1. Create a `datetime` object representing July 4, 2025.
2. Calculate the number of days from today until July 4, 2025. Print the result.

Part 3: Date Formatting

1. Retrieve the current date.
2. Format the date into the form "DD-MM-YYYY" (e.g., "08-10-2023"). Print the result.
3. Format the current date to display the full weekday name, day, month name, and year (e.g., "Sunday, October 8, 2023"). Print the result.